MPI Performance Evaluation with Different Workload Sizes and Process Counts in a Single-Machine WSL2 Environment

Yunlong Wang[1], Griffith College Dublin, BSc (Hons) in Computing Science
2057646043@qq.com

***Abstract*—Parallel computing enables a computational workload to be divided among multiple processes, thereby effectively reducing computation time. However, the efficiency gains achieved by increasing the number of processes depend on both the workload size and the execution environment. This paper investigates how the number of processes and the workload size affect the performance of MPI-based parallel computing within a single-machine WSL2 environment. We evaluated performance across different workload sizes—10 million, 50 million, and 100 million iterations—using 1, 2, 4, and 8 MPI processes. Each configuration was executed five times, with performance assessed based on average computation time, speedup, and parallel efficiency. The results indicate that increasing the number of processes reduces computation time across all three workload sizes; however, the observed speedup falls short of ideal linear scaling. For the 10 million-iteration workload, the speedup achieved with 8 processes was approximately 3.9989, corresponding to a parallel efficiency of about 49.99%. For the 50 million-iteration workload, the speedup achieved with 8 processes was approximately 4.97, with a parallel efficiency of about 62.11%. These findings demonstrate that the performance gains derived from process-level parallelism vary according to the workload size and the number of processes employed. This study provides an empirical baseline for evaluating the effects of workload size and process count on the performance and scalability of MPI-based computing.**

***Index Terms*—MPI, parallel computing, parallel efficiency, speedup.**

I. INTRODUCTION

Parallel computing can reduce execution time by dividing a computational workload among multiple processes. However, the speedup obtained depends on the workload and the number of processes used [1], [3].

Parallel computing allows a computational workload to be divided among multiple processes so that parts of the computation can be performed in parallel. MPI (Message Passing Interface) is commonly used to develop parallel programs. However, using more processes does not always lead to a proportional reduction in computation time. The performance improvement may depend on the size of the workload and the environment in which the program runs.

In this study, I investigate how workload size and the number of MPI processes affect computation time and parallel performance. The experiment was conducted in a single-machine WSL2 (Windows Subsystem for Linux 2) environment. Three workload sizes were tested: 10 million, 50 million, and 100 million iterations. Each workload was executed using 1, 2, 4, and 8 MPI processes, with five runs for each configuration. Mean computation time, speedup, and parallel efficiency were used to compare the results.

The results show that using more MPI processes reduced computation time for all three workload sizes. However, the speedup was lower than the ideal linear speedup, and the results differed between workload sizes. For example, using eight processes achieved a speedup of approximately 4.97 for the 50-million-iteration workload, while the parallel efficiency was approximately 62.11%. These results show that workload size should be considered when evaluating the performance of MPI parallel computing.

This study provides experimental results on MPI performance in a single-machine WSL2 environment. It focuses on the effects of workload size and process count on computation time, speedup, and parallel efficiency. The findings can serve as a starting point for further experiments with different workloads and execution environments.

II. RELATED WORK

MPI performance is commonly evaluated using execution time, speedup, and parallel efficiency. Pacheco [1] explains how these metrics can be used to study the performance of parallel programs and compare different process configurations. Ideally, increasing the number of processes should reduce execution time proportionally. In practice, the speedup depends on the workload and the execution environment, so experimental measurements are needed to understand the actual performance.

The reliability of these measurements is also important. Hunold and Carpen-Amarie [2] discuss the challenges of designing reproducible MPI benchmarks and show why

experimental conditions and measurement methods must be carefully considered. Their work highlights the importance of controlling experimental factors when comparing MPI performance.

Compared with studies that focus on MPI communication operations or large-scale parallel applications, this study uses a simple computational workload in a single-machine WSL2 environment. It examines three workload sizes and four process configurations, with five repetitions for each configuration. The aim is to compare how workload size and process count affect computation time, speedup, and parallel efficiency under the tested conditions. The experiment does not aim to measure multi-node communication performance or to generalise its results to all MPI applications.

Pacheco [1] introduces the basic principles of parallel programming and the evaluation of parallel performance. Amdahl [3] explains why the benefits of parallel processing are limited by the sequential part of a program. These concepts provide a theoretical background for analysing speedup and parallel efficiency.

The reliability of performance measurements is also important. Hunold and Carpen-Amarie [2], [4] discuss experimental design and reproducibility in MPI benchmarking. Their work highlights the importance of carefully designed experiments and statistically meaningful measurements. The Open MPI documentation [5] also provides practical recommendations for conducting performance benchmarks. In this study, these principles motivate the use of repeated runs, a warm-up execution and a clearly defined timing procedure.

A further consideration is the distinction between computational speedup and communication overhead. In MPI programs, the observed execution time can be influenced by both the work performed by each process and the coordination required to combine partial results. The balance between these components depends on the program structure and the size of the workload. For this reason, results from a simple computational kernel should be interpreted as a controlled case study rather than as a complete representation of all MPI applications. The present experiment focuses on process count and workload size while keeping the basic computation consistent across configurations.

III. Experimental Methodology

A. Experimental Environment

The experiment was conducted on a laptop running Windows 11 with Ubuntu under WSL2. The MPI program was written in C, compiled using GCC, and executed with Open MPI. The system was equipped with an Intel Core i9-13900HX processor, with 32 logical processors reported by WSL2. The software environment included Ubuntu 26.04.1, GCC 15.2.0, and Open MPI 5.0.10. All experiments were performed within the same environment to maintain consistency across different workload sizes and MPI process configurations. These hardware and software specifications provide relevant context for interpreting the measured computation times and parallel performance. Table I summarises the main experimental environment.

TABLE I
EXPERIMENTAL ENVIRONMENT

Component	Configuration
Processor	Intel Core i9-13900HX
Operating environment	Windows 11 with WSL2
Linux distribution	Ubuntu 26.04.1
Compiler	GCC 15.2.0
MPI implementation	Open MPI 5.0.10
Logical processors reported by WSL2	32

B. Workload and Experimental Design

A C program was developed to evaluate the computational performance of MPI parallel processing. The program calculates a sum by distributing the iteration range among MPI processes. Each iteration contributes a value determined by the iteration index modulo 100. MPI processes calculate their assigned portions, and the program uses MPI reduction to verify the final result.

Three workload sizes were tested: 10 million, 50 million, and 100 million iterations. For each workload, the program was executed with 1, 2, 4, and 8 MPI processes. Each configuration was tested five times, resulting in 60 experimental runs across the 12 configurations.

The same computational task was used for all configurations to make their performance results comparable. A warm-up execution was performed before the timed repetitions. The reported computation time represents the maximum measured computation duration among the MPI processes. The timing excludes the subsequent MPI reduction operation; therefore, it measures the computational phase rather than the total execution time of the MPI program.

C. Performance Metrics

Three metrics were used to evaluate the performance of the MPI program: mean computation time, speedup, and parallel efficiency. The mean computation time was calculated from five runs for each configuration. The standard deviation was also calculated to measure the variation between runs.

Speedup was calculated as the ratio of the mean computation time using one process to that using (p) processes:

$$S_p = \frac{T_1}{T_p}$$

where T_1 and T_p represent the mean computation times using one process and (p) processes, respectively.

Parallel efficiency was calculated as:

$$E_p = \frac{S_p}{p} \times 100\%$$

where S_p is the speedup and p is the number of MPI processes. Speedup indicates the performance improvement achieved by parallel processing, while parallel efficiency shows how effectively the processes are used compared with ideal linear scaling.

IV. Results And Discussion

A. Computation Time

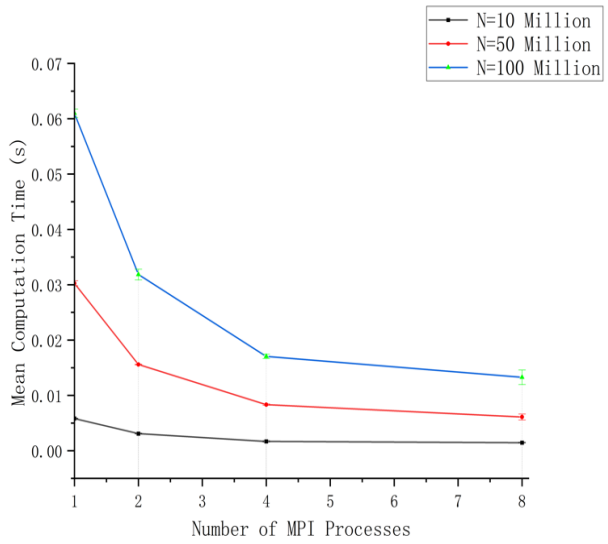


Fig. 1. Mean computation time for different workload sizes and MPI process counts.

The mean computation time decreased as the number of MPI processes increased for all three workload sizes. For the 10-million-iteration workload, the mean computation time decreased from approximately 0.00583 s with one process to 0.00146 s with eight processes. For the 50-million-iteration workload, it decreased from approximately 0.03026 s to 0.00609 s. The 100-million-iteration workload showed a similar trend, with the mean computation time decreasing from approximately 0.06098 s to 0.01328 s.

These results show that increasing the number of processes reduced the measured computation time in all tested configurations. However, the improvement was not proportional to the increase in process count. The standard deviation across repeated runs also varied between configurations, indicating that measurement variability should be considered when interpreting the results.

B. Speedup and Parallel Efficiency

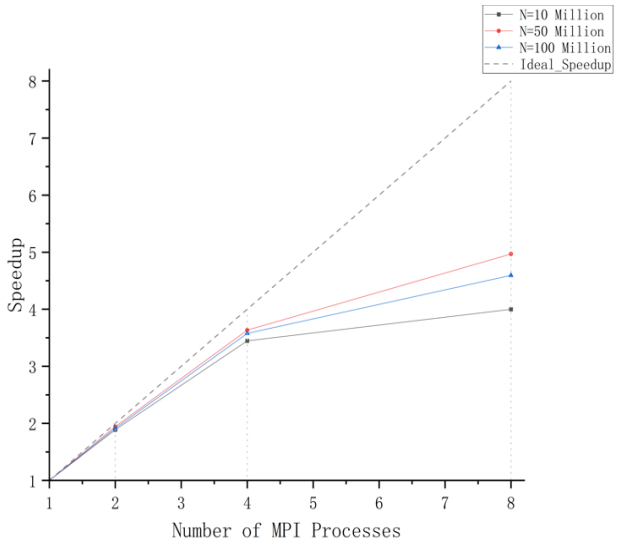


Fig. 2 compares the measured speedup for the three workload sizes with ideal linear scaling.

Fig. 2. Speedup for different workload sizes, compared with ideal linear scaling.

The speedup and parallel efficiency results provide further information about the performance of the MPI program. With eight processes, the 10-million-iteration workload achieved a speedup of approximately 4.00, corresponding to a parallel efficiency of 49.99%. For the 50-million-iteration workload, the speedup was approximately 4.97 and the parallel efficiency was 62.11%. The 100-million-iteration workload achieved a speedup of approximately 4.59, with a parallel efficiency of 57.41%.

The results show that the 50-million-iteration workload achieved the highest speedup and parallel efficiency among the three workloads when using eight processes. However, the largest workload did not produce the highest speedup. Therefore, the results do not indicate a simple linear relationship between workload size and parallel performance. Within the tested configurations, both workload size and process count affected the measured speedup and efficiency.

Overall, the experiment demonstrates that adding MPI processes can reduce computation time, but the performance gain does not increase proportionally with the number of processes. These findings apply to the tested computational kernel and single-machine WSL2 environment. Further experiments would be needed to determine whether similar trends occur with other workloads or execution environments.

V. Conclusion

This study evaluated the effects of workload size and MPI process count on computational performance in a single-machine WSL2 environment. Three workload sizes and four process configurations were tested, with five runs for each configuration. The results showed that increasing the number of processes reduced computation time across all tested

workloads, although the speedup remained below ideal linear scaling. Among the tested eight-process configurations, the 50-million-iteration workload achieved the highest speedup of approximately 4.97 and a parallel efficiency of 62.11%. These findings show that the performance benefits of MPI parallelism vary with workload size and process count. Future work could extend the evaluation to additional workloads and execution environments, including multi-node systems, to investigate how these factors affect parallel performance.

REFERENCES

[1] P. S. Pacheco, An Introduction to Parallel Programming. Burlington, MA, USA: Morgan Kaufmann, 2011.

[2] S. Hunold and A. Carpen-Amarie, "MPI Benchmarking Revisited: Experimental Design and Reproducibility," arXiv:1505.07734, 2015.

[3] G. M. Amdahl, "Validity of the single processor approach to achieving large scale computing capabilities," in Proceedings of the Spring Joint Computer Conference, pp. 483–485, 1967, doi: 10.1145/1465482.1465560

[4] S. Hunold and A. Carpen-Amarie, "Reproducible MPI Benchmarking is Still Not as Easy as You Think," IEEE Transactions on Parallel and Distributed Systems, vol. 27, no. 12, pp. 3617–3630, 2016, doi: 10.1109/TPDS.2016.2539167

[5] Open MPI Project, "Benchmarking Open MPI applications," Open MPI 5.0.1 Documentation. [Online]. Available: https://docs.open-mpi.org/en/v5.0.1/tuning-apps/benchmarking.html